Mix Counters, Not Matrices, for Laptop English

Author: Nikunj Mehndiratta. 30 October 2026.

No weight-free model family has yet produced coherent English. The evidence therefore supports a weight-free conversational *system* on a Core Ultra 7 255U laptop, but not a weight-free open-domain *generator*. One family has a language-grade benchmark result: context-mixing compressors. These models are hashed counters and bit-history tables combined by a small linear mixer that learns online. Starting from nothing, the pure-counter cmix reaches **about 1.17 bits per character on enwik8**, level with the 46M-parameter LSTMs of 2016–17. The current Hutter Prize record runs under rules of one core, under 10 GB of RAM and about 50 hours, which is close to this laptop's budget. The other weight-free families top out much lower. Weightless lookup-table nets, learned logic-gate networks, Tsetlin machines and hyperdimensional codes stop at classification, at character-level demos with no metric, or at **4.39 BLEU** on translation. Lookup layers (Memory Layers, Engram, Over-Encoding) and retrieval (kNN-LM, infini-gram) help only when added to a dense backbone, and their own authors report that they do not improve open-ended generation and can make it worse. The recommended design, **Grove-CM**, places a byte-level context-mixing language model, conditioned on dialogue features, inside the team's existing chat front end: the noisy-channel autocorrect, the intent router and the reply bank. Grove holds the dialogue state and computes every number exactly. The design must give up "no dense weights" in one place: the mixer, which has thousands to low millions of weights, each trained by a single-layer local gradient. It gives way in a second place only if the experiments force it: a 1–20M-parameter neural component for multi-sentence coherence. Every published coherent generator keeps dense attention or recurrence, and the 2026 compression frontier itself added small transformers to its mixers. A five-rung benchmark ladder tests each step against a transformer trained for the same number of laptop-hours, using the team's measured 1,350–2,040 tokens/s for a 19M stack. The counters should win the low rungs, where one fast pass is enough. Rung 2, story-level consistency, is the deciding test, and its kill rule hands the generator role to a dense hybrid. The whole plan needs about twelve laptop-days of compute and six to eight calendar weeks for two people **[estimate]**.

> **How to read the numbers. [measured]** marks an experimental result from a published paper's tables or a public benchmark (the LTCB and Hutter Prize lists). These ran on hardware other than the 255U unless stated. **[measured, team]** marks the team's own runs on the 255U; team results without a link are the team's own runs, not published elsewhere. **[author]** marks a claim made in a README, an abstract or a projection, or quoted by a secondary source, without a protocol our notes could check. **[vendor]** marks a company's figure about its own product. **[estimate]** marks our arithmetic or inference, which nothing has measured. Compressor "bpc" is cumulative and online unless stated: the model learns while it scores. That is not the same protocol as a neural model's held-out score, and the experiment plan corrects for it.

Six weight-free families, one language-grade result

Two questions hide inside the brief. The first is whether any model without dense weight matrices has learned English well enough to talk. The second is whether such a model can be trained cheaply on a 15 W-class laptop CPU. The survey asks every family the same thing: what is the hardest English task it has demonstrably done, and what stopped it from going further? The table gives the answer in brief. The sections that follow give the evidence.

Family	Hardest language result found	Label	What limits it on language	CPU training evidence
Weightless / LUT nets (WiSARD, ULEEN, DWN, PolyLUT)	None; best is keyword spotting 71.5%	[measured]	Fixed binary inputs, no sequence mechanism, 57.5% on CIFAR-10 shows weak composition	None reported; FPGA and microcontroller results only

Family	Hardest language result found	Label	What limits it on language	CPU training evidence
Learned logic-gate nets (difflogic and successors)	WMT'14 En→De 4.39 BLEU after discretisation	[measured]	Depth collapse, discretisation gap, no attention	Training needs CUDA; CPU Python path 50–100× slower [author]
Tsetlin machines	IMDb 91.27%; word embeddings that beat GloVe; char-level Shakespeare samples with no metric	[measured] / [author]	One class per output symbol; fixed Boolean window	Bitwise and integer; "minutes to hours on a desktop CPU" [author]
Hyperdimensional computing	21-language identification at 97.8%	[measured]	Superposition noise; acts as a hashed n-gram profile	One pass of additions
Lookup and memory layers (Memory Layers, PEER, Engram, Over-Encoding, MoLE)	+3.0 MMLU over an iso-FLOP MoE (Engram-27B)	[measured]	Only work as add-ons to a dense backbone	No CPU training run reported
Retrieval and n-gram stores (kNN-LM, RETRO, NPM, infini-gram)	∞-gram 47% next-token agreement; −34% perplexity when interpolated with GPT-2	[measured]	Does not improve, or hurts, open-ended generation	Count-based or index-based; no gradient training
Context-mixing compressors (PAQ, cmix, Hutter entries)	enwik9 0.73–0.80 bpc; enwik8 1.17 bpc online	[measured]	Sampling with learning on loops; frontier now adds small transformers	Single core, under 10 GB, 1–2 days per GB
Brain-inspired and local learning (HTM, Hopfield/SDM, FF, PC, SNN, reservoirs, ES)	Every large success keeps dense weights trained by backprop	[measured] / [author]	No benchmark-grade result without dense weights	Mostly GPU, or iteratively costlier than backprop

Weightless, logic-gate and Tsetlin networks stop at classification

Modern weightless networks are now trained by gradient descent through relaxed lookup tables. On images and tables they are excellent for their size. DWN reaches **98.31% on MNIST, 89.12% on FashionMNIST and 76.3% on jet substructure**, but only **57.51% on CIFAR-10**, against 80.10% for a binarised CNN **[measured]** (DWN). ULEEN raised the weightless MNIST record from 91.5% to 98.46% **[measured]** (ULEEN). Their headline speed figures come from FPGAs (125 ns and 19 nJ per MNIST sample) and an 8-bit microcontroller. The DWN paper does not even state its training hardware (DWN). The ablation shows that most of the gain comes from learning which input bits feed which table, and the CIFAR-10 gap shows that a fully connected table network handles compositional input poorly. Language is the most compositional input there is. The notes found **no weightless or LUT network applied to any text or sequence task**.

Differentiable logic-gate networks are the strongest weight-free image classifiers. Each neuron learns which of the 16 two-input Boolean functions to be, and the trained network compiles to plain C. The convolutional version reaches **86.29% on CIFAR-10 with 61 million gates [measured]** (NeurIPS 2024), and the original paper claims more than a million MNIST images per second on one CPU core **[author]** (difflogic). Training is the problem on this laptop. The repository requires CUDA, and its authors say the CUDA kernel is **50–100× faster than the CPU Python path** for large models **[author]** (difflogic GitHub). The only sequence result is a recurrent logic-gate encoder-decoder on WMT'14 English→German. It scored **5.00 BLEU during training and 4.39 BLEU after discretisation**, against 5.41 for a GRU baseline that is itself weak. Competitive systems on that task score 20–30 or more **[measured]** (RDDLGN). A July 2026 preprint reports that logic-gate networks "don't reliably get better with more depth" because of optimisation collapse **[author]** (arXiv 2607.21633). Depth is exactly what language composition needs.

Tsetlin machines are the most genuinely CPU-native learners in the survey. They learn conjunctive clauses over Boolean literals using integer automata, so both training and inference reduce to AND, zero-test and vote counting. On bag-of-words IMDb sentiment they score **87.2%, or 91.27% with Drop Clause**, against 84.4% for XGBoost **[measured]** (Drop Clause). A Tsetlin autoencoder learns interpretable word embeddings that

outperform GloVe on six classification tasks [measured] (TM embedding). The language-modelling attempts are all at character level. Tsetlin.jl generates Shakespeare-like text after 400+ epochs but reports no perplexity, size or time. It claims MNIST training for 1,000 epochs in 5 seconds and 38M predictions per second on an unnamed "desktop CPU" **[author]** (Tsetlin.jl). Energy claims of 52× **[vendor]** (eeNews Europe) and "up to 10,000×" come from a company that was still "finalizing external benchmarking" **[vendor]** (Forbes). Independent academic work does find more than an order of magnitude lower inference energy than binarised networks on a microcontroller **[measured]** (ETHEREAL). The structural limit is the output. A Tsetlin language model is a classifier with one clause bank per output symbol, so 65–100 characters are tractable and a 30K-subword vocabulary has never been shown **[estimate]**.

Hyperdimensional and brain-inspired learners stop at demos

Hyperdimensional computing encodes character trigrams by permuting and binding random 10,000-dimensional vectors, then sums them into a profile. It identifies 21 languages with **97.8% accuracy** after a single pass of additions **[measured]** (random-indexing language ID). The notes found **no hyperdimensional next-token model with a reported perplexity**. In the LLM era, the method's main language role is probing transformer states (Hyperdimensional Probe). A bundled prototype is in effect a hashed n-gram count with fixed-capacity noise, so its ceiling should sit below a Kneser-Ney n-gram model **[estimate]**.

Brain-inspired learning has the same pattern in a sharper form: every success at scale kept dense weights trained by backprop. HTM sequence memory learns online with local permanence updates and matched an LSTM on streaming taxi-demand data **[measured]** (HTM sequence memory). Its word-level demonstrations are qualitative and report no perplexity (Semantic Folding). Numenta's large CPU speedups (up to 17× against an A100 on BERT-Large at batch 1, shrinking to 2.2× at batch 8) belong to a sparse, backprop-trained transformer product running on Xeon AMX **[vendor]** (Intel). The 255U has neither AMX nor AVX-512 (Intel ARK). Associative memory succeeded only inside transformers. The modern Hopfield update *is* softmax attention (Hopfield Networks is All You Need), and linear attention is a Hebbian fast-weight memory (linear transformers as fast-weight programmers). In both cases the projections that make retrieval generalise are dense and trained by backprop. Forward-Forward's only language demonstration is next-character prediction on 248 strings from Aesop's Fables (Forward-Forward). Predictive coding trains small transformer language models "comparable to backprop" **[author]**, but it keeps dense weights and costs many relaxation steps per sample (predictive coding for transformers). Target propagation and feedback alignment fall further behind backprop as tasks get harder **[measured]** (biologically motivated learning at scale). The spiking models SpikeGPT and SpikingBrain-76B are backprop-trained or converted from dense checkpoints (SpikeGPT; SpikingBrain). Reservoirs train only a readout and are much faster than transformers, but predict characters worse at matched trainable parameters **[measured]** (reservoir vs transformer, 2025). Even then, the readout is a dense matrix. EGGROLL's evolution strategies pre-trained integer-only recurrent language models without backprop **[author]**, but they needed GPU-scale forward-pass throughput (EGGROLL). ADIOS induced grammars from child-directed speech, but never produced a probabilistic generator (PNAS 2005).

Lookup layers multiply a dense backbone instead of replacing it

The 2024–2026 lookup results are real and large, and every one of them sits on a dense backbone. Meta's memory layers swap one to three feed-forward layers for a product-key table. Gains hold up to about three swapped layers, and removing more dense FFNs hurts. At matched compute the memory model matches a dense model with about **2× the FLOPs** on QA **[measured]** (Memory Layers at Scale). DeepSeek's Engram looks up hashed 2- and 3-gram suffixes of normalised token IDs. Validation loss is U-shaped in the split of the sparse budget, with the optimum at about **20–25% in tables and the rest in MoE experts**. Engram-27B beats an iso-parameter, iso-FLOP MoE by **+3.0 MMLU, +5.0 BBH and +2.2 GSM8K**, and adding slots lowers loss log-linearly at no extra compute **[measured]** (Engram). Over-Encoding lets a **400M model match a 1B baseline's training loss** by adding hashed n-gram input embeddings **[measured]** (Over-Tokenized Transformers). Even the GPT-2-small speedrun record now carries an **84.6M-row hashed n-gram table** (modded-nanogpt). Two neighbouring ideas are not weight-free at all. MatMul-free LMs keep dense ternary matrices and trail Transformer++ by 0.8–1.8 average points (MatMul-free LM). T-MAC's lookup kernels speed up inference of low-bit weight matrices, and its own repository says it "cannot guarantee significant speedups on x86" **[vendor]** (T-MAC).

For a CPU, the important property is how these tables train. Engram's index depends only on token IDs, so the table's gradient is a sparse scatter-add into a few rows per token, with no dense matmul in its backward pass. Engram trains its tables with Adam at 5× the base learning rate and no weight decay (Engram). That makes a hashed n-gram table the cheapest trainable parameter available on a laptop. Its cost is memory bandwidth and optimizer state, not FLOPs. A million fp32 rows at d = 256 take about 1 GB, plus about 2 GB of dense Adam state **[estimate]**. Two gaps matter for the plan. No paper reports CPU training of any lookup-augmented model, and the smallest published results (MoLE at 160M, Over-Encoding at 150–400M) are an order of magnitude above laptop scale (MoLE).

Retrieval and infini-gram copy spans but cannot hold a thread

A datastore can stand in for knowledge parameters. RETRO's gains are comparable to about **10× more parameters [measured]** (RETRO), and kNN-LM cut WikiText-103 perplexity to 15.79 without extra training **[measured]** (kNN-LM). But neither replaces the model that writes the text. kNN-LM and similar interpolations **do not improve open-ended generation** by MAUVE or human evaluation, because retrieval entropy grows as generated queries drift **[measured]** (kNN-LM generation study). NPM fills masks from a corpus well enough to beat GPT-3 zero-shot on closed-set tasks, but it cannot generate, and it needs about **70 GB of RAM** for Wikipedia **[measured]** (NPM). Infini-gram is the purest weight-free language model in the survey: a suffix array with unbounded-order backoff. It agrees with the human next token **47% of the time**, and over 75% when a suffix of 16 or more tokens matches, against 29% for a 5-gram. Interpolated with GPT-2 1.6B, it cuts perplexity by 34%. Its authors still call it "not ready to replace neural LMs", because it can derail open-ended generation **[measured]** (infini-gram). Its costs suit a laptop: about **7 bytes per token** on disk and 10–180 ms per CPU query. The FM-index version shrinks the index to about 44% of corpus size (infini-gram mini).

Context mixing is the only weight-free family with a benchmark-grade language number

Compressors are language models scored by bits per byte, and the best CPU-only ones are strong. On 24 July 2026, fx2-cmix-T set the Hutter Prize record at **100,424,672 bytes for enwik9**, about 0.80 bpc including the decompressor **[measured]**. The prize rules allow about 50 hours, under 10 GB of RAM and a single core with no GPU (Hutter Prize). Epoch AI notes that June–July 2026 submissions tripled the total improvement since 2019, to about 15% (Epoch AI). On the Large Text Compression Benchmark, CPU entries reach **0.73–0.78 bpc on enwik9** in 9.4–10.2 GB of RAM at 4.4–9 KB/s. One of them, fractal-zip, ran on a Core i7-11850H laptop **[measured]** (LTCB). For scale, Chinchilla 70B compresses enwik9 to about 8.3% raw, roughly 0.66 bpc **[measured]** (Language Modeling Is Compression). On enwik8, cmix v21 reaches **1.170 bpc learning online**, while Transformer-XL reaches 0.99 and a 44M 12-layer transformer 1.11 after offline training. The 46M LSTMs of 2016–17 sit at 1.24–1.27 **[measured]** (LTCB; Transformer-XL). Classical n-grams sit far lower: a Kneser-Ney 5-gram has perplexity **67.6 on One Billion Word**, against 21.8 for Transformer-XL **[measured]** (One Billion Word benchmark). Context mixing closes most of that gap by combining hundreds of such models.

The mechanism explains both its strength and why it is not strictly weight-free. Each context model hashes a context (order-n bytes, the current word, sparse skips, and so on) into a slot holding a one-byte nonstationary bit-history state. An adaptive table maps that state to a probability, with updates at about 1/4096 of the error. Hash tables are laid out to cost at most **three cache misses per byte per model** (Data Compression Explained). A small neural network, the mixer, then weights the models' predictions in context (PAQ). The learned state is therefore gigabytes of counters plus a modest set of dense mixer weights, and every update is local: a counter increment, a state transition or a one-layer gradient step on coding cost. The recent frontier is telling. Several of the top 2026 entries ("fx2-cmix-transformer", "cmix-lex-transformer" and the record-holding fx2-cmix-T) add a small transformer running on one CPU core as one more input to the mixer (LTCB). The best pure counter-based reference point is about the cmix v21 level, 0.86 bpc on enwik9 **[measured]**.

Generation is the weak spot, and it is where this design must work hardest. Any compressor can be sampled, but gzip continuations are gibberish (Language Modeling Is Compression). The cmix author says cmix "is not good at text generation" because learning cannot be switched off, so samples "spiral into repetitive cycles". His successor, gmix, freezes learning during generation and uses a gated linear network as its mixer. It is early-stage, with no published samples (gmix). The only conversational precedent is MegaHAL. It used two fourth-order Markov models, generated candidate replies from the user's keywords, returned the most "surprising" one and learned from every user utterance (How MegaHAL works). Nobody has published a quantitative evaluation of text generated by a strong context mixer, so how coherent ~0.8–1.2 bpc counter models are when sampled is an open question that the plan below has to answer.

The laptop rewards counting over multiplying at small budgets

The 255U has AVX2 but no AVX-512 or AMX, a 12 MB L3, and two channels of LPDDR5/X-8400 or DDR5-6400 (Intel ARK). That is roughly 100–134 GB/s of peak bandwidth **[estimate]**. The team measured fp32 matmul at **434 GFLOP/s** and a dropout-free 19M transformer stack at **1,350–2,040 tokens/s on battery**, with no embedding or LM head in that measurement **[measured, team]**. That works out to **4.9–7.3M tokens per laptop-hour [estimate]**. Training a 19M model to its Chinchilla-optimal 380M tokens would take 52–78 hours, and one epoch of enwik8's 90M training characters would take 12–18 hours **[estimate]**. The table puts the candidate learners on the same footing.

Learner on the 255U	Data learned per laptop-hour	Resident memory	Generation or scoring cost	Basis
19M transformer, 12 threads	4.9–7.3M tokens	~0.3 GB with AdamW	—	[measured, team] rate; memory [estimate]
5M character transformer	~15–30M chars	<0.1 GB	—	[estimate], 3–4× the 19M rate
Hutter-class pure CM, **one core**	~16–32 MB	9.4–10.2 GB	110–230 µs/byte	LTCB desktop speeds [measured] → [estimate]

Learner on the 255U	Data learned per laptop-hour	Resident memory	Generation or scoring cost	Basis
Strong PPM (ppmonstr-class), one core	~1 GB	1.7 GB	~3.6 µs/byte	LTCB [measured] → [estimate]
Fast CM/PPM (lpaq9m, ppmd), one core	~4 GB	0.26–1.5 GB	~0.9 µs/byte	LTCB [measured] → [estimate]
Hashed n-gram table, 1M rows × 256 fp32	set by its backbone	~1 GB + ~2 GB Adam	one row gather per n-gram	[estimate]
Infini-gram suffix array, 1B tokens	build in hours	~7 GB on SSD	10–180 ms/query (~10–50 tok/s)	[measured] cost per token → [estimate]
Tsetlin machine (Tsetlin.jl)	MNIST, 1,000 epochs in 5 s	small	38M predictions/s	[author], CPU unnamed

The table points to a hypothesis. On character data, a single core running a Hutter-class mixer learns from **2–6× more bytes per hour** than the whole chip can feed a 19M transformer, and a fast PPM learns from about 500× more **[estimate]**. Cramming found that, at a fixed small budget, transformer type and size barely change final loss, because "per-token efficiency … depends strongly on model size, but not transformer type" (Cramming). So a weight-free design cannot expect to learn more *per token* than a transformer. Its one realistic lever is **tokens per CPU-second**, and counters have that lever. The published 1.06–1.11 bpc transformer figures came from multi-epoch GPU training. A laptop transformer given a day sees less than two epochs of enwik8. A matched-laptop-hours comparison is therefore the counter models' best chance, and testing it is cheap. The price is memory. Hutter-class mixers hold about 10 GB against 0.3 GB for the transformer, so every comparison must report stored bytes, and the plan includes an equal-memory arm.

The bit-level families fit AVX2 less well than their papers suggest. Tsetlin clauses and hypervector similarity reduce to AND, XOR and popcount, but AVX2 has no vector popcount instruction. Counting goes through scalar `popcnt` or a `vpshufb` nibble lookup, and that step bounds clause voting **[estimate]**. A DWN LUT-6 is 64 bits and stays in L1, but the efficient path evaluates 256 samples in parallel per register. That favours batched classification over the one-token-at-a-time loop of generation **[estimate]**. Difflogic and DWN train through floating-point relaxations and gain nothing from bit tricks during training. Context mixers, by contrast, were designed for exactly this regime: one core, cache-miss-bound hash tables, and nothing worth vectorising except the mixer's dot products. The hybrid E-core layout matters less for them, because each model is one thread. Up to ten configurations can run side by side for sweeps, but whether single-threaded CM keeps P-core clocks on Windows has to be measured: the team's E1 run found that pinning lowered clocks for PyTorch **[measured, team]**.

A protocol trap sits under every compressor comparison. Compressor bpc is online and cumulative, so it includes the learning curve and also benefits from adapting to the test text. Neural enwik8 numbers come from a held-out split scored with frozen weights (Language Modeling Is Compression). The notes found no frozen held-out bpc for cmix or PAQ. The plan therefore gates on **frozen held-out bpc for every arm** and reports online bpc only as a secondary figure. Online bpc stays relevant for conversation, where a model that adapts to each user is a feature.

Grove-CM: a ranked set of candidate designs

Three candidates follow, ranked by expected value under the "no dense weights" constraint. Rank 1 is the design to build. Rank 2 is the fallback, planned in advance, that the kill rules switch to. Rank 3 contains cheap probes whose likely outcome is a supporting role. All three reuse the team's pieces. On the team's frozen 264-message set, the Stage 1 talk bot routes **85.2%** of messages correctly (baseline 33.0%), detects out-of-scope messages 82% of the time, answers at **3.1 ms p95** and repeats no replies. The noisy-channel autocorrect fixes **61.3% of typos** (Norvig's corrector on the same counts: 43.5%), changes 0.04% of clean words and costs about 3 ms per word **[measured, team]**. Grove executes step programs exactly and refuses rather than guesses.

Rank 1: Grove-CM keeps the language in counters and the meaning in programs

Grove-CM splits each turn into problems with a known answer and problems with only a probable answer. Exact components handle the first kind and counters handle the second. No component trains by backprop through depth, and the only dense weights are in the mixer.

Layer	Component	Learned state	Dense weights?	Training cost	Per-turn cost	Evidence for the role
Input	Noisy-channel autocorrect, extended to take its word prior from the CM's word model	Word and bigram counts, error model	None	Built	~3 ms/word; less if gated to out-of-vocabulary tokens	[measured, team]
Understand	Router: rules, then a classifier. Strict variant: Tsetlin clauses or hyperdimensional prototypes	Clauses or hypervectors; logistic weights today	Today: a tiny logistic layer. Strict variant: none	Minutes	<5 ms	TM text classification (Drop Clause); HDC language ID (random-indexing language ID)
State and content	Grove step programs and a dialogue frame; evolved policy trees gated by checkers	Programs	None	Evolution against checkers	Milliseconds	Team reports
Select	Reply bank with BM25 or hypervector retrieval, persona filter, no-repeat memory	Index	None	Minutes	1–20 ms [estimate]	XiaoIce hybrid (XiaoIce)
Score and generate	**CM-LM**: byte, word, sparse and dialogue-context models; adaptive probability maps; a gated linear mixer; frozen base plus per-conversation cache	1–8 GB of counters	**Mixer only**	One pass: minutes (fast tier) to ~1.3–2.5 days per GB (top tier) [estimate]	~1 µs/byte (fast tier) to 110–230 µs/byte (top tier)	LTCB, gmix
Long memory (optional)	Infini-gram or FM-index over the dialogue corpus	Suffix array	None	Hours [estimate]	10–180 ms/query	infini-gram
Guard	Grove re-parses every number in the output; persona contradiction check	—	None	—	<1 ms [estimate]	E2E semantic control (E2E NLG Challenge)

The new component is **CM-LM**, a byte-level context mixer built as a language model rather than a compressor. It has three changes from a compressor. First, besides the usual order-1 to order-8 byte contexts, word contexts and sparse skip contexts, it hashes **dialogue-context models**: the keywords of the previous user turn, the router's intent label, the persona slot in play and the position in the turn, each combined with the current partial word or order-n context. These are what let the mixer condition a reply on the conversation, in the same way PAQ conditions on arbitrary hashed features. This is a design proposal with no published test **[estimate]**. Second, it runs at two speeds. A frozen base model is trained offline on licence-clean dialogue such as SODA (CC BY 4.0, card) and Synthetic-Persona-Chat (CC BY 4.0, card). A small cache model per conversation learns online, **from user turns only**, to avoid the self-reinforcing loops the cmix author describes (gmix). Third, it comes in two tiers. A fast lpaq-class tier at about 1 µs per byte does scoring, reranking and autocorrect priors. A slower, stronger tier is used offline and for the rare free-generation fallback.

CM-LM works in three modes, ordered from safest to riskiest, and each turn uses the safest mode that succeeds. In **scoring mode**, retrieval proposes candidate replies from the bank, and CM-LM ranks them by log-probability under the dialogue-context models. Fifty 80-byte candidates cost about 4 ms on the fast tier **[estimate]**. In **infill mode**, a template carries Grove's values locked in their slots, and CM-LM chooses among paraphrases and connectives. In **free-generation mode**, used only when the bank abstains, CM-LM works MegaHAL-style: it samples several short candidates from the user's keywords with learning frozen, reranks them, and passes them through Grove's numeric check and the persona filter. A 500-byte reply costs about 50–100 ms on the top tier **[estimate]**. This ordering follows XiaoIce's lesson. In Microsoft's ratings, retrieval alone was unacceptable 35.0% of the time, generation alone 66.6%, and the hybrid that ranked both 23.1% **[vendor]** (XiaoIce). Generation earns its place by patching coverage, not by carrying the conversation.

The expected ceiling is narrow-domain conversation with retrieval-quality fluency, exact numbers and per-user adaptation, plus short generated replies that are locally fluent. **Multi-sentence coherence from CM-LM alone is not expected**, because nothing in a counter model carries state across sentences beyond hashed context. That state lives in Grove's dialogue frame instead, which is why the frame, not the language model, owns the conversation.

Rank 2: admit a 1–20M dense component only where coherence demands it

If rung 2 of the ladder shows that counters cannot keep a story consistent, the smallest admissible compromise is a small neural model, and the evidence for that step is unusually consistent. Every coherent generator in the survey keeps dense attention or recurrence. Meta found that replacing more than about three FFN layers with memory hurts (Memory Layers). Engram's optimum leaves 75–80% of the sparse budget in experts (Engram). The 2026 compression record was set by adding a small transformer to a context mixer (Hutter Prize; LTCB). TinyStories puts the coherence threshold low: consistency appeared when hidden size rose from 64 to 128 with at least two layers, and models of **8.3–28M parameters scored 5–9 out of 10 on consistency** in GPT-4 grading **[measured]** (TinyStories).

Two variants should be compared. **Rank 2a** puts a 1–5M LSTM or transformer *inside* CM-LM as one more mixer input, following fx2-cmix-T. It keeps vocabulary and phrase knowledge in counters and asks the network only for composition. **Rank 2b** inverts the balance. A 5–20M transformer backbone, near the 9M Chinchilla extrapolation for a 15-hour run from the team's earlier report, carries Engram-style hashed 2- and 3-gram tables of 1–2M rows at d = 128–256. The tables are trained with a row-sparse optimizer, ideally Adagrad-style row state to keep memory under 4 GB **[estimate]**. Rank 2b doubles as the **equal-memory baseline** that a fair comparison against a 10 GB counter model needs. Both variants keep Grove, the router and the bank unchanged. Neither escapes dense weights, but both keep the dense part to a few tens of megabytes, which is the honest minimum the literature supports.

Rank 3: Tsetlin, logic-gate and hyperdimensional generators are probes, not plans

Three strictly weight-free generators deserve one cheap run each, mainly because nobody has published a bits-per-character figure for any of them. A **Tsetlin character model** trains on bitwise and integer operations, with thermometer-coded or hashed n-gram literals over a fixed window. A **recurrent logic-gate network** is small enough (around 100K gates) to train on a CPU at all, given the 50–100× penalty of the Python path **[author]**. A **hyperdimensional or sparse-distributed-memory next-character model** uses XOR-and-popcount retrieval over context codes. Their likely ceiling is the hashed n-gram they approximate **[estimate]**, so the probe's question is narrow: can any of them beat an order-3 PPM at equal laptop-minutes? Their stronger roles are already in the Rank 1 table: Tsetlin clauses or hypervector prototypes as a strictly weight-free router, and hypervector or SDM codes as a retrieval index. Reservoirs, HTM, Forward-Forward, predictive coding and spiking models are not ranked. Each either keeps a dense trained matrix, costs more than backprop on serial hardware, or has no language metric at all.

Where "no dense weights" has to bend

The compromises should be stated plainly so that nobody mistakes the design for something purer than it is. **First, the mixer is unavoidable.** Every PAQ-family model and every leaderboard entry uses a learned mixer to weight its context models (PAQ; gmix). Its size is thousands to low millions of weights **[estimate]**. It is trained online by a single-layer gradient on coding cost, never by backprop through a stack. **Second, the router's logistic layer is avoidable.** It is a few dozen intents by a few hundred features, and a Tsetlin or hypervector router can replace it if strictness matters more than the last points of accuracy. **Third, a 1–20M neural component is conditional.** It enters only if rung 2's kill rule fires, and only in the Rank 2a or 2b form. **Fourth, "weight-free" never meant "parameter-free".** A Hutter-class mixer stores about 10 GB of learned counters, about as many bytes as a 2–5B-parameter fp16 model **[estimate]**. Every comparison must count them. What never bends in Rank 1 is the training regime: counting, state transitions and local one-layer updates, with no stored activations and no backward pass.

A five-rung ladder decides it in about twelve laptop-days

The gate rules come first, because they make the results comparable. Every rung compares the weight-free arm with a **transformer trained on the same data for the same laptop-hours** on the same 255U, plugged in, on a performance power plan, with dropout off and PyTorch's default 12 threads. The team's battery runs drifted by up to 27% within a quarter-hour **[measured, team]**, so arms are interleaved and the baseline is repeated at the end of each block. CM arms run single-threaded, so they may use up to ten parallel processes in the same wall-clock time, and they report core-hours as well. All gates use **frozen held-out** scores. Every arm reports stored bytes and bytes touched per token alongside quality, and training data is deduplicated against test data, since large tables memorise. An arm **climbs** a rung when it matches the matched transformer (within 0.03 bpc, or within one point on judged scores) or beats it. Literature figures set the "credible" bar, not the gate. Two seeds per transformer arm estimate run-to-run noise. All thresholds below are our proposals, not sourced standards.

ID	Rung and test	Weight-free arms	Matched baseline	Climb if	Kill if	Laptop compute	People
W0	Calibration	Build CM-LM: fast, mid and top tiers, freeze switch, scoring API, dialogue contexts. Time each tier on enwik8 on the 255U	Re-measure 1M, 5M and 19M char transformers *with* embedding and head, on mains (also finishes E1's mains half)	Top tier within 2× of LTCB desktop speeds	A 1 GB top-tier pass would exceed 5 days → top tier becomes offline-only and the mid tier takes over	~1 day	3–5 person-weeks for the engine (C++, Python binding)
W1	Rung 0: shakespeare_char, held-out bpc	PPM orders 2–8; CM-LM tiers; KN n-gram; Rank 3 probes (Tsetlin, small recurrent logic net, HDC/SDM)	nanoGPT CPU config (2.71 bpc in 3 min, nanoGPT) and a tuned 1M model at 3, 15 and 60 minutes	CM-LM ≤ transformer at 15 and 60 min	A Rank 3 arm worse than order-3 PPM at equal minutes is dropped	~0.5 day	1 week, shared with W0
W2	Rung 1: enwik8 (90/5/5), frozen bpc, online bpc secondary	CM-LM fast, mid and top tiers	5M and 19M char transformers, checkpoints at 2, 8 and 24 h	Parity at 8 h and 24 h; credible ≤1.23, the cmix row in Transformer-XL	CM-LM >0.10 bpc worse at 24 h *and* the gap grows from 8 h to 24 h → CM-LM loses the generator role (keeps scoring) and Rank 2 is promoted	~3 days	—

ID	Rung and test	Weight-free arms	Matched baseline	Climb if	Kill if	Laptop compute	People
W3	Rung 2: TinyStories subset (50–100M tokens); 50 stories generated with learning frozen; LLM judge on grammar, consistency and creativity, each pair run in both orders	CM-LM (top tier); Rank 2a if W2's kill rule fired	TinyStories-style transformer at equal hours (8–20 h)	Consistency within 1 point of matched; grammar ≥7. Credible: consistency ≥8 (28M class, TinyStories)	Mean consistency ≤3 (the 1M-model level) while matched ≥6 → free generation passes to Rank 2a or 2b; CM-LM stays as scorer and infiller	~2 days; judge costs a few dollars	0.5 week
W4	Rung 3: BabyLM strict-small (10M words), BLiMP zero-shot, scored by sentence log-probability	CM-LM trained on 10M words	Transformer at 4 h and 20 h	≥ matched and ≥0.63 (the 2023 OPT-125M baseline). Credible ≥0.80 (BabyLM 2023)	BLiMP <0.63 → CM-LM scores are not used to judge grammaticality in reranking	~0.5 day; can run alongside W3	0.5 week
W5	Rung 4: whole dialogue system. Held-out bpb on Synthetic-Persona-Chat and SODA; ACUTE pairs (ACUTE-Eval), 40 per axis; failure log; numeric fidelity; p95 latency	Grove-CM (Rank 1), or Grove + Rank 2 if promoted	(a) Stage 2 retrieval-only talk bot; (b) the same front end with a matched-hours transformer generator	Wins ≥27 of 40 pairs against (a) on at least one axis and loses none; zero numeric mismatches reach users; p95 <500 ms	Fails against (a) → ship retrieval plus templates; CM-LM survives only as reranker and autocorrect prior	~3 days (mid tier on 0.5–1 GB of dialogue; top tier once)	2–3 weeks, including human judging
W6	Rank 2b ablation (can run any time after W0)	—	5M transformer ± hashed 2/3-gram tables (1–2M rows), dense Adam vs row-sparse update (the earlier E7)	Tables cut held-out loss by ≥0.02 nats at equal laptop-hours	<0.02 nats with ≥1M rows → drop tables, keep the plain small model	~2 days	1 week

The ladder's order follows the gate rule in the benchmark notes: do not climb until the design matches a same-laptop, same-hours transformer on the current rung. W4 is cheap and can run beside W3. W6 is independent and fills idle laptop time. The 27-of-40 bar in W5 comes from a two-sided 5% binomial test on 40 blinded pairs, as in the team's conversation report. An LLM judge is acceptable for triage only if every pair runs in both orders, and it never grades numbers: Grove does. Dataset and tool downloads (enwik8, TinyStories, BabyLM, SODA, Synthetic-Persona-Chat, any compressor source code) need approval before W0. Compressor code should be built from source after checking its licence, never run from prebuilt binaries.

Writing the predictions down now keeps the ladder honest **[estimate, registered predictions]**. At rungs 0 and 1, CM-LM should match or beat the matched transformer up to about 8 laptop-hours, because the transformer will not have finished one epoch of enwik8. The gap should narrow by 24 hours, and the 19M transformer may overtake. At rung 2, CM-LM should write grammatical sentences (grammar 5–7) and lose the thread across them (consistency 2–4), which would fire the kill rule and promote Rank 2a. At rung 3, BLiMP should land below the 0.70 baselines, since n-gram-like models see local agreement but not long-distance dependencies. At rung 4, Grove-CM should beat the retrieval-only bot on coverage and fallback rate rather

than on judged quality. If rung 2 instead passes, the result would be new: the first evidence that a counter model can be coherent at story level. It would justify writing up the weight-free design itself.

Costs add up as follows. Compute is about **twelve laptop-days**: W0 1, W1 0.5, W2 3, W3 2, W4 0.5, W5 3 and W6 2 **[estimate]**. People cost is about **7–9 person-weeks**, of which the CM-LM engine is the largest item. Two people should finish in **six to eight calendar weeks**, with compute running overnight beside engineering **[estimate]**. The early exits are cheap. If W2's kill rule fires in the second week, the pure-counter generator has cost about four laptop-days and the engine, and the engine still pays for itself as the system's scorer, infiller and autocorrect prior.

Conclusion

"Weight-free" turns out to be the wrong axis for this design. The one that predicts success is **how a model learns**: by counting and one-layer local updates, or by backprop through depth. Counting already reaches 2016–17 LSTM quality in bits per character on one CPU core. It learns in a single pass, adapts online to each user, and fits the laptop's memory. What counting has never shown is composition across sentences, and on that point the evidence is unanimous across lookup layers, retrieval, logic gates and compressors: each one's best language result either sits on a dense network or was beaten by one. Both frontiers have reached the same hybrid from opposite ends. Compressors added small transformers to their mixers, and LLMs added hashed n-gram tables to their transformers. For this project, the useful question is therefore where the boundary between counting and composing sits for a laptop-sized conversational system, not whether dense weights can be eliminated.

The plan's main contribution is a test that weight-free designs usually avoid: a same-laptop-hours transformer, frozen held-out scoring, and stored bytes counted. That test is also the counters' best chance, because on this chip a transformer is short of tokens, not of capacity. Whatever rung the crossover lands on, the result is useful. Grove-CM ships either as a fully counter-based conversational system with exact semantics, or as the same system with a few megabytes of dense composition placed exactly where the measurements show it is needed. Grove's guarantees are the same in both cases: numbers are exact, state is explicit, and the system abstains when it is unsure. The open question is only how much of the surface text counters can write.